

Developing Generative AI Solutions

Capstone Project and Assessment Guide

Final English Edition

Submission date	Thursday, 6 August 2026 - exact cutoff and Google Form link will be shared soon.
Presentation schedule	To be announced later.
Pass mark	60% or higher.
Project format	Group project - team size will be announced later.
Optional deployment bonus	+5 bonus points for a working published URL.
Expected product	A working Python-based generative AI product that can be demonstrated locally or online.

Main objective. Build a focused product that solves a real problem and connects the main course concepts in one understandable flow: language model prompting, RAG, an agent with at least two tools, and a working application. Public deployment is optional. A working published URL earns +5 bonus points.

1. Purpose of the Capstone Project

The capstone is the practical outcome of the course. It is not expected to be a large commercial platform. It should be a small but complete product that a real user can open, test, and understand.

Your team should demonstrate that it can:

1. call a language model from Python using a clear prompt and structured output;
2. retrieve relevant evidence from a project knowledge source through RAG;
3. allow an agent to select and use at least two meaningful Python tools;
4. present the result through a working application that can run locally or online; and
5. explain the system choices, limitations, failures, and basic safety considerations.

Freedom of choice. Teams may select any useful project idea, model provider, embedding model, vector database, agent framework, interface, or deployment service. Free and paid services are both allowed. Marks are awarded for correct, useful, and understandable implementation - not for choosing an expensive tool.



2. Assessment Structure

Assessment component	Weight	Minimum evidence expected
Theory Exam	30%	Understanding of the theoretical slide content from all course days.
RAG	25%	A complete, visible retrieval pipeline using embeddings and a vector database.
Agent and Tools	25%	An agent that selects and uses at least two meaningful Python tools.
Day 1 Concepts	10%	A clear prompt, Python model call, structured result, and appropriate error handling.
Group Presentation	5%	A clear explanation and a live demonstration of the working product.
Documentation and GitHub	5%	A complete repository with all code and a useful README file.
Total	100%	A final score of 60% or higher is required to pass.
Optional Deployment Bonus	+5	A working published URL. Core rubric: 100 points; maximum raw score: 105 points.

The table above contains the complete grading categories. The expectations in this guide describe the minimum evidence for each category; they are not additional sub-percentages.

Theory Exam (30%). Understanding of the theoretical slide content from all course days.

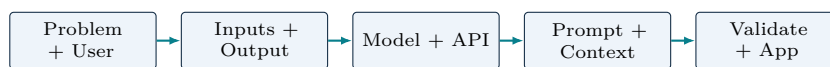
Practical Condition

The project must be runnable and have a working demonstration. A notebook, design mock-up, or presentation without a functioning product is not enough. Public deployment is **not mandatory**. Teams that publish a working URL receive **+5 bonus points**. A local application is fully acceptable for the required 100% rubric, but it must run correctly during the presentation.

3. Minimum Practical Requirements

Day 1 Concepts - 10%

Use the Day 1 **model-to-solution** process as a simple planning guide:



Review the Day 1 slides for extra guidance, especially the difference between a model and a complete solution, the system layers, and the development lifecycle.

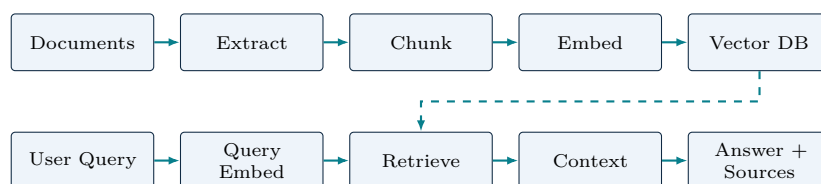
The product should show a simple model-to-application flow:

- The main logic is written in Python.
- The application calls an LLM API or an approved local model.
- The prompt defines the role, task, limits, and expected output.
- The output is organized for the product: JSON, table, checklist, report, or defined fields.
- Missing input, unsupported files, insufficient evidence, and model errors are handled clearly.
- The team can describe one improvement made to the prompt after testing.

RAG - 25%

At minimum, the code should contain:

- a useful knowledge source;
- document or data ingestion;
- extraction and basic cleaning;
- a reasonable chunking method;
- an embedding model;
- a vector database or store;
- query embedding and retrieval;
- metadata such as source/page/section;
- a grounded prompt using retrieved evidence;
- visible sources when possible; and
- a safe response when evidence is missing.



Agent and Tools - 25%

- The agent has at least **two meaningful Python tools**.
- Each tool has a clear purpose and defined inputs and outputs.
- The agent selects Tool 1, Tool 2, or both according to the request or intermediate result.
- Python validates arguments before tool execution.
- Tool results return to the agent or product pipeline.
- The agent has a step limit, stopping rule, timeout, or another simple control.
- Tool failures do not crash the full application.
- High-impact actions are confirmed or safely simulated.

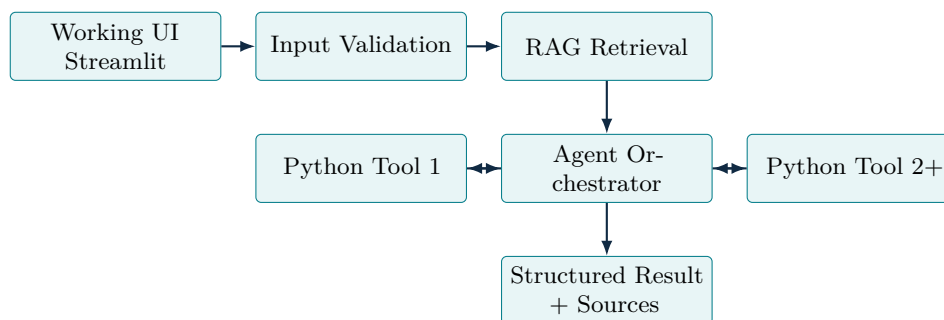
RAG retrieval does not count as one of the two required tools. The project must show both RAG and at least two additional useful tools. The tools should solve real parts of the problem rather than exist only to satisfy the rubric.

Examples include calculating a score, exporting a report, generating a study plan, creating a safe draft ticket, ranking suppliers, normalizing dates, or querying an approved structured dataset.

4. Product Architecture, Technology, and Data

Recommended Product Architecture

A simple implementation is enough. Keep the main concepts visible in the code.



Suggested Technology Options

Component	Easy recommended option	Freedom allowed
Language	Python	Python should remain the main language for product logic.
Interface	Streamlit	Gradio or another interface is acceptable.
LLM	Gemini API	Another hosted API or an approved local model.
Embeddings	Gemini embedding model	A local sentence-transformers model or another API.
Vector database	Qdrant Cloud	Supabase pgvector, Chroma, Pinecone, Weaviate, or another suitable vector store.
Validation	Pydantic	Clear manual validation is acceptable for a small project.
Optional deployment	Streamlit Community Cloud	Render or another compatible service; a working published URL earns +5 bonus points.

Lowest-complexity path: Python + Streamlit + Gemini + Qdrant Cloud + GitHub. For the optional deployment bonus, Streamlit Community Cloud is the recommended publishing service.

Qdrant Cloud currently offers a free prototype cluster; Supabase pgvector is useful when a team also needs Postgres tables, storage, or user-related data. Gemini currently offers free-tier options for supported models and embeddings. Service limits can change, so teams should verify current limits before the demo.

Data Options

Use a small, relevant, clean dataset. Teams may use public documents, open datasets, team-created sample files, synthetic data, or a combination of PDF, text, CSV, image, and audio when useful.

If real data is difficult to obtain, synthetic data may be generated for the project. State this clearly in the README. Never upload confidential, personal, sensitive, or copyrighted data without permission.

Optional Deployment Recommendation

For a Python Streamlit project, Streamlit Community Cloud is the default recommendation because it deploys directly from GitHub and supports secrets through the deployment settings. Render is allowed when the project needs a custom Python web service or a different architecture. Test the same Python version locally and in deployment, keep dependencies in `requirements.txt`, and store secrets outside the repository.

Official references: [Gemini API pricing](#); [Qdrant Cloud](#); [Supabase pgvector](#); [Streamlit Community Cloud](#); [Render free services](#).

5. Suggested Code Structure and README

Optional Python Project Structure

```
project-name/
|- app.py
|- requirements.txt
|- README.md
|- .env.example
|- .gitignore
|- data/sample_documents/
|- scripts/ingest_documents.py
|- src/config.py
|- src/schemas.py
|- src/llm/client.py
|- src/llm/prompts.py
|- src/rag/loaders.py
|- src/rag/chunking.py
|- src/rag/embeddings.py
|- src/rag/vector_store.py
|- src/rag/ingest.py
|- src/rag/retriever.py
|- src/agent/orchestrator.py
|- src/agent/tool_registry.py
|- src/agent/guardrails.py
|- src/tools/tool_1.py
|- src/tools/tool_2.py
|- src/services/product_pipeline.py
'- evaluation/test_cases.json
```

This structure is not mandatory. A smaller project may combine files, while a more complex project may add modules. The important point is that the instructor can quickly find the prompt logic, RAG pipeline, agent, tools, application, and test evidence.

Documentation and GitHub Repository - 5%

No separate report is required. The `README.md` file is the main documentation and should contain:

1. Project title.
2. Team name and members.
3. Problem and target user.
4. Solution and expected value.
5. Main features.
6. Simple architecture or flow.
7. Day 1 implementation.
8. RAG pipeline explanation.
9. Embedding model and vector database.
10. Agent design and two or more tools.
11. Technology stack.
12. Data sources and synthetic-data disclosure.
13. Local setup and run instructions.
14. Required variables in `.env.example`.
15. Published application URL, if available (optional bonus).
16. Known limitations and future improvements.
17. Sample tests or screenshots when useful.

The repository must contain all source code and dependency files. Do not commit real API keys, passwords, tokens, or private data. AI coding assistants may be used, but the team must understand the code and be able to explain the main process.

Simple evaluation evidence. Include a few test cases showing: a normal request, missing evidence, Tool 1, Tool 2, a request using both tools when appropriate, and one handled failure.

6. Five Suggested Real-World Project Ideas

These are suggestions only. Any useful idea is accepted if it meets the assessment requirements.

1. Meeting-to-Action Assistant

Problem: Decisions and responsibilities are lost after meetings.

Input: Meeting transcript plus project guidelines or team information.

RAG: Retrieves roles, policies, deadlines, and approved task formats.

Possible tools: Normalize dates; export action items to CSV or a formatted report.

Output: Decisions, tasks, owners, deadlines, risks, and a downloadable action plan.

2. Requirements and Document Compliance Checker

Problem: Reviewing a proposal against requirements is slow and inconsistent.

Input: Requirements document and submitted proposal.

RAG: Retrieves requirements and supporting evidence from both documents.

Possible tools: Calculate compliance score; export a compliance report.

Output: Compliant, partial, or missing requirements with sources.

3. Smart Lecture Study Assistant

Problem: Students need a faster way to study long slides and transcripts.

Input: Lecture slides, transcript, and a student question.

RAG: Answers from approved lecture material with slide/page references.

Possible tools: Generate flashcards; create a study plan or quiz file.

Output: Grounded answers, summary, concepts, questions, and study resources.

4. Procurement Offer Comparison Assistant

Problem: Comparing supplier offers with technical and purchasing rules takes time.

Input: Supplier offers, requirements, and evaluation criteria.

RAG: Retrieves exact requirements, specifications, and supplier evidence.

Possible tools: Calculate weighted scores; generate a comparison report.

Output: Comparison table, gaps, risks, scores, and recommendation.

5. Maintenance Inspection Assistant

Problem: Technicians need quick access to relevant procedures during initial inspection.

Input: Equipment description or image, symptoms, and a maintenance manual.

RAG: Retrieves relevant manual sections and safety instructions.

Possible tools: Calculate a risk level; create a maintenance ticket or checklist.

Output: Possible issue, evidence, warning, inspection steps, and structured ticket.

Choose a problem your team can finish and demonstrate. A focused solution that works well is stronger than a large unfinished system.

7. Submission and Presentation

Submission

The complete project is due on **Thursday, 6 August 2026**. The exact cutoff and Google Form link will be shared soon.

The form is expected to collect:

- team name and member names;
- project title and short description;
- complete GitHub repository URL;
- published application URL, if available (optional bonus); and
- any short note required by the instructor.

Ensure the repository is accessible to the instructor and includes the complete code used for evaluation. Before submission, run the project locally. If the project is published, test the published URL as well. Confirm that required files, setup instructions, environment-variable names, dependencies, and sample data are present.

Group Presentation - 5%

The presentation date, exact time, and team schedule will be announced later.

Expected duration	Approximately 5 minutes.
Maximum slides	10 slides.
Questions	Approximately 1-2 short questions per team.
Presenters	One member or multiple members may present.
Required demo	A live demonstration of the working product, locally or through a published URL.
Team understanding	The team should understand the essential product flow even if one member presents most of it.

Suggested Presentation Flow

1. Team and project title.
2. Real-world problem and target user.
3. Proposed solution and value.
4. Input data and knowledge base.
5. System architecture.
6. RAG pipeline.
7. Agent and available tools.
8. Live product demonstration.
9. Test result, limitation, or failure case.
10. Conclusion and future improvement.

Test the demo shortly before the presentation. Keep a small approved sample input ready. If an external service is temporarily unavailable, be prepared to show the code path and a recent screenshot or recorded output, but the normal expectation remains a live working demo.

What Makes the Project Useful?

The product should create a result that a target user can act on: a sourced answer, comparison, checklist, score, report, plan, task file, or safe draft action. Avoid building a generic chatbot with no clear user, data, workflow, or outcome.

8. Questions, Final Checklist, and Portfolio Value

Ten Questions Your Team Should Be Ready to Answer

1. **What problem does the product solve and who is the user?** State the pain point and benefit clearly.
2. **Why is an LLM useful here?** Identify the task requiring understanding, extraction, classification, generation, or reasoning.
3. **Why did you use RAG?** Explain what project-specific evidence the model needs.
4. **How was the data prepared?** Mention extraction, cleaning, chunking, metadata, and synthetic data.
5. **Why this embedding model and vector database?** Give a practical reason such as simplicity, cost, or language support.
6. **How does the agent select tools?** Explain the descriptions, schemas, and decision flow.
7. **Can you show Tool 1, Tool 2, and a combined request?** Demonstrate that the choice depends on the task.
8. **What happens when evidence is missing?** Show that the product avoids unsupported confident claims.
9. **How did you test errors and failures?** Mention invalid input, failed retrieval, and tool errors.
10. **What is the main limitation and next improvement?** Give an honest, realistic answer.

Final Team Checklist

Use this checklist to track progress from idea selection to the final presentation.

Project Planning

- ☐ Clear real-world problem and target user.
- ☐ Defined input, output, and useful value.
- ☐ Manageable idea and team responsibilities.

Day 1 - Model to Solution

- ☐ Python main logic.
- ☐ Clear model call, prompt, context, and structured output.
- ☐ Input validation and basic error handling.
- ☐ Team can explain model-to-solution flow.

RAG

- ☐ Relevant knowledge or synthetic sample data.
- ☐ Extraction, cleaning, chunking, embeddings, and vector storage.
- ☐ Retrieval with source metadata.
- ☐ Grounded answer and safe missing-evidence response.

Agent and Tools

- ☐ At least two meaningful Python tools.
- ☐ Clear inputs, outputs, and validation.
- ☐ Agent selects Tool 1, Tool 2, or both.
- ☐ Tool failures and stopping conditions handled.

Repository and Documentation

- ☐ Project runs locally.
- ☐ Dependencies, `.env.example`, sample data, and instructions included.
- ☐ No secrets or sensitive data in GitHub.
- ☐ README covers team, idea, architecture, RAG, agent, tools, data, setup, tests, and limitations.
- ☐ Repository is accessible to the instructor.

Presentation and Optional Publishing

- ☐ No more than 10 slides.
- ☐ Approximately five-minute explanation and live demo rehearsed.
- ☐ Team can answer basic questions.
- ☐ Optional published URL works for the +5 bonus points.

Build for learning and for your portfolio. After the course, improve the README, add screenshots and results, and publish the project as a portfolio item or LinkedIn post. A well-explained product with a working demo can be more valuable than a much larger unfinished idea.

Make it useful. Make it work. Make it understandable.